Dự án nhỏ: Quản lý sinh viên

Áp dụng OOP xây dựng chương trình quản lý sinh viên.

Mục tiêu bài học

- Áp dụng OOP để xây dựng chương trình quản lý sinh viên
- Kết hợp lớp, danh sách, hàm và vòng lặp trong một dự án nhỏ
- Rèn kỹ năng chia chương trình thành các phần nhỏ

1. Yêu cầu bài toán

Xây dựng chương trình console quản lý danh sách sinh viên với các chức năng: thêm sinh viên, hiển thị danh sách, tìm theo tên, xoá sinh viên và thống kê điểm trung bình.

2. Thiết kế

- Lớp `SinhVien` lưu mã, tên, điểm và có phương thức `xep_loai()`.
- Lớp `QuanLySinhVien` chứa danh sách (list) các sinh viên và các phương thức xử lý: `them`, `hien_thi`, `tim_theo_ten`, `xoa`, `diem_trung_binh`.
- Hàm `main()` hiển thị menu, nhận lựa chọn của người dùng bằng vòng lặp `while`.

3. Cài đặt mẫu

```
class SinhVien:
    def __init__(self, ma, ten, diem):
        self.ma, self.ten, self.diem = ma, ten, diem

    def xep_loai(self):
        return "Giỏi" if self.diem >= 8 else "Khá" if self.diem >= 6.5 else "Trung bình"

class QuanLySinhVien:
    def __init__(self):
        self.ds = []

    def them(self, sv):
        self.ds.append(sv)

    def tim_theo_ten(self, tu_khoa):
        return [sv for sv in self.ds if tu_khoa.lower() in sv.ten.lower()]

    def diem_trung_binh(self):
        return sum(sv.diem for sv in self.ds) / len(self.ds) if self.ds else 0
```

4. Gợi ý mở rộng

Lưu danh sách ra file (văn bản hoặc JSON) để dữ liệu không mất khi tắt chương trình; kiểm tra dữ liệu nhập (điểm từ 0 đến 10, mã không trùng); sắp xếp danh sách theo điểm.

Tóm tắt

- Chia bài toán thành các lớp có trách nhiệm rõ ràng.
- Dùng list để lưu tập đối tượng, duyệt bằng vòng lặp hoặc list comprehension.
- Menu bằng `while` giúp chương trình chạy liên tục đến khi người dùng thoát.

Bài tập vận dụng

1. Hoàn thiện chương trình với đầy đủ menu và các chức năng ở trên.
2. Thêm chức năng sắp xếp danh sách theo điểm giảm dần.
3. Thêm chức năng ghi và đọc danh sách sinh viên từ file JSON.

Học thêm, bài tập và hướng dẫn giải

Python — Sổ tay học thêm và thực hành tổng hợp

Tài liệu tự biên soạn bổ sung cho chín bài Python của khóa. Đọc sau phần chính; không cần học thuộc mọi ví dụ. Mục tiêu là giải thích được kết quả và tự kiểm tra chương trình.

Mục tiêu học thêm

- Phân biệt phép gán, sao chép và thay đổi đối tượng
- Kiểm tra dữ liệu đầu vào và các giá trị biên
- Tổ chức chương trình quản lý sinh viên bằng hàm và lớp

1. Biến tham chiếu tới đối tượng

Phép gán không luôn tạo bản sao. Với danh sách, hai tên có thể cùng trỏ tới một đối tượng:

```
a = [7, 8]
b = a
b.append(9)
print(a)  # [7, 8, 9]
c = a.copy()
c.append(10)
print(a)  # [7, 8, 9]
```

copy() là sao chép nông: danh sách ngoài mới nhưng đối tượng lồng bên trong vẫn có thể dùng chung. Với danh sách số như ví dụ, sao chép nông đủ để append không ảnh hưởng danh sách gốc. Đây là lỗi thường gặp khi tạo nhiều danh sách điểm từ cùng một mẫu.

2. Điều kiện, vòng lặp và giá trị biên

Với xếp loại điểm, kiểm tra đầu vào trước khi phân loại. Đặt điều kiện điểm cao trước khi dùng elif theo ngưỡng giảm dần để không nuốt các trường hợp sau.

```
def classify(score):
    if not 0 <= score <= 10:
        raise ValueError("Điểm phải từ 0 đến 10")
    if score >= 8.5:
        return "Tốt"
    if score >= 7:
        return "Khá"
    if score >= 5:
        return "Đạt"
    return "Cần ôn"

assert classify(5) == "Đạt"
assert classify(8.5) == "Tốt"
```

range(1, 4) tạo 1, 2, 3, không có 4. Khi kiểm tra vòng lặp, thử danh sách rỗng, một phần tử và nhiều phần tử. Với while, xác định biến nào làm điều kiện thay đổi; nếu không thay đổi, vòng lặp có thể không kết thúc.

3. Hàm thuần và lỗi đối số mặc định

Hàm tính toán nên trả kết quả, còn nhập/xuất thực hiện ở lớp giao diện. Nhờ vậy có thể kiểm tra bằng assert mà không phải nhập tay. Tránh đối số mặc định là danh sách nếu mỗi lần gọi cần một danh sách mới.

```
def add_student(name, students=None):
    if students is None:
        students = []
    students.append(name)
    return students

assert add_student("An") == ["An"]
assert add_student("Bình") == ["Bình"]
```

Nếu dùng students=[] trong định nghĩa hàm, danh sách được tạo một lần khi định nghĩa, nên các lần gọi có thể giữ dữ liệu cũ. None giúp tạo danh sách mới cho mỗi lần không truyền đối số.

4. Lớp, kế thừa và thiết kế dự án

Student nên chịu trách nhiệm về một sinh viên; danh sách và thao tác tìm kiếm nên thuộc lớp quản lý hoặc hàm quản lý. Dùng kế thừa khi có quan hệ “là một”, không dùng chỉ để tránh viết vài dòng giống nhau.

```
class Student:
    def __init__(self, student_id, name, scores):
        if not scores or any(not 0 <= s <= 10 for s in scores):
            raise ValueError("Cần điểm hợp lệ")
        self.student_id = student_id
        self.name = name.strip()
        self.scores = list(scores)

    def average(self):
        return sum(self.scores) / len(self.scores)

s = Student("SV01", "An", [8, 6])
assert s.average() == 7
```

Danh sách điểm được sao chép để append vào danh sách bên ngoài không tự thay đổi điểm sinh viên. Trong ứng dụng hoàn chỉnh còn cần kiểm tra tên rỗng và mã trùng khi thêm vào hệ thống.

Bài tập theo ba mức

1. Dễ: dự đoán a sau khi b = a rồi b.append(5); giải thích khác biệt với b = a.copy().
2. Trung bình: viết hàm nhận danh sách điểm, trả trung bình hoặc None nếu rỗng; không làm tròn trong quá trình cộng.
3. Khó: thiết kế quản lý sinh viên với thêm, cập nhật, tìm theo mã và xếp loại. Khi mã trùng hoặc điểm ngoài khoảng, không được thay đổi dữ liệu cũ.

Gợi ý lời giải và tiêu chí đánh giá

1. a và b dùng chung danh sách nên cùng nhìn thấy phần tử mới. Với copy, append chỉ thay danh sách b.
2. Kiểm tra if not scores trước khi chia; dùng sum(scores) / len(scores). Thử [], [0], [10], [6, 8].
3. Lưu sinh viên trong dict theo mã để phát hiện trùng. Kiểm tra dữ liệu trước khi ghi; tách validate, add và update. Kiểm thử mã không tồn tại, tên trắng, điểm -1, điểm 11 và sửa sinh viên đang có. Bài đạt khi có thể giải thích cấu trúc dữ liệu và chứng minh cập nhật lỗi không làm mất bản ghi.

Học tiếp và tự đánh giá

Tự viết thêm chức năng lưu JSON, xử lý lỗi file và đọc lại dữ liệu. So sánh dữ liệu trước và sau một vòng lưu/đọc. Không dùng eval để đọc dữ liệu người dùng. Trước khi chuyển sang framework, cần chạy được chương trình bằng terminal và có các kiểm tra giá trị biên.

Nguồn đối chiếu: https://docs.python.org/3/tutorial/datastructures.html ; https://docs.python.org/3/tutorial/controlflow.html ; https://docs.python.org/3/tutorial/classes.html